

explained. It is imperative to remember that duplicate table names across databases added to the configuration would raise a warning because the two tables would get duplicate class names during Codegen. Typically, Codegen will raise an error when such issues arise. To counter this, set prefix and/or suffix values for `classNames` in `codegen_settings.xml` for the database whose ORM class files you would want to carry the prefix/suffix.

Location of files

The location of files generated during the Codegen process depends on the type of file. Here is a list of locations where the Codegen files are stored:

1. **includes/meta_controls/generated/:** The files in this directory contain the actual code for MetaControls and DataGrids (in MetaControl-Gen and DataGrid-Gen classes) and are overwritten each time Codegen runs. They should not be used to store any custom logic for the generated MetaControls and DataGrids.
2. **includes/meta_controls/:** This directory will contain the class files where you can write your custom logic for the MetaControls and DataGrids because they are created once and are not overwritten on subsequent code generations. These are directly derived from the MetaControlGen classes.
3. **includes/model/generated/:** Contains ORM-Gen class files for each table with the functions we discussed earlier. They are overwritten on subsequent code generations and these too cannot hold any custom code you write in them.
4. **includes/model/:** Contains the ORM class files that you should use while designing your application. They are not overwritten and are one great place to store your custom methods for each table separately!
5. **includes/formbase_classes_generated/:** Contains the abstract classes to derive from, for creating pages to list and edit entries in tables, updated on each Codegen run.
6. **drafts/panels/:** Contains Codegen panels that can be directly used for creating, editing and deleting entries in tables. They use MetaControls for the respective tables and add the 'Save', 'Cancel' and 'Delete' buttons for CRUD functionality. These panels are reusable in your application, and are recreated on each Codegen.
7. **drafts/:** Contains individual pages for CRUD functionality for tables. They are overwritten each time Codegen runs.

In addition to the files created to work upon tables, the QCBued Codegen system also updates three special files located in `includes/model/generated`. The files are: `_class_paths.inc.php`, `_type_class_paths.inc.php` and `QQN.class.php`. These files are vital to the autoloading feature in QCubed, and must not be tampered with.

Using the generated code

You have got ORM classes, draft forms, panels, MetaControls and DataGrids. Where and how to use them is, of course, up to you and the design of the app. However, there are a few recommendations that will help you along the way:

- Do not ever alter any files which are over-written on Codegen. You would only lose your work.
- Use ORM class files to write any code that works on table data directly. This will facilitate organisation of code, and reduce the number of custom files you need to create; e.g., if you want to create a method that loads objects using a non-indexed column, the ORM class is where you should define it.
- Choose drafts pages and the AJAX panel over `phpPgAdmin` (or `phpMyAdmin`) for inputting data—they are safer and come with validation logic.
- If you want to show a control on your Web page, which directly relates to a table in the DB, try to use MetaControls rather than creating a new control, time and again. You can alter the looks and behaviour of input controls you get via MetaControls. If you alter the columns (e.g., alter the length of `varchar` fields), Codegen will update the constraints on the MetaControls, making sure you do not have to rewrite the logic.
- Feel free to copy the MetaControls, Panels, Controls and Pages generated by Codegen and create your own classes by modifying them—they are there to help you, not to restrict you. However, do check for changes in schema since your custom files are not automatically updated. If you simply want to 'add' a functionality, it is better to create a new class inheriting from the generated class (much the same way ORM classes are inherited from ORM-Gen classes).
- Directory locations are also controlled using variables defined in `configuration.inc.php`. If you want to change their location, you can.
- Though QCubed gives you the freedom to write your custom queries, do it when `QQuery` does not satisfy your needs perfectly. Try to stick to the generated classes wherever you can—it saves labour.
- DataGrids should be used to create simple tables with basic search functionality.
- DataGrids are not SEO friendly—they never provide a hyperlink to the second (and subsequent) page results; instead, work using actions and events. Use your own pagination if you require SEO.
- Your draft pages can be used to change data. Set `ALLOW_REMOTE_ADMIN` to 'False' in the configuration file to make sure that access to them is restricted.

All the above are suggestions, not rules. If you have a reason to, you can violate them (e.g., when you want to access the draft pages on your website from a remote computer, you should set `ALLOW_REMOTE_ADMIN` to your public IP address). Use Codegen as an aid and it will reduce about 70 per cent of your workload straight away—you must have already realised how! 

By: Vaibhav Kaushal

The author is a 25 year old college dropout from Bangalore who also happens to be a core contributor to QCubed. He loves writing for technology magazines and his blog when he is not busy fiddling around with QCubed or developing his website (<http://www.c-integration.com/>).